

Evolving RL Algorithms: The GECCO 2026 Paper

Reinforcement learning algorithms are conventionally defined by hand-designed update rules that map experience to parameter changes. While automated discovery has been applied to adjacent components of RL systems, including neural architecture search [ZophLe2017], hyperparameter tuning [Xu2018], and reward shaping [Ma2024], the learning update rule itself has remained largely fixed. A small number of prior works have explored learning the update rule directly. Oh et al. [Oh2020] optimized differentiable update functions, while Co-Reyes et al. [CoReyes2021] evolved structured symbolic loss expressions via genetic programming. However, these methods operate over restricted, parameterized representations of the learning rule, such as differentiable update networks or symbolic loss templates, and therefore cannot express fundamentally different training procedures as executable code. Sygkounas, Loutfi, and Persson [Sygkounas2026] address this gap directly in their paper “Evolutionary Discovery of Reinforcement Learning Algorithms via Large Language Models,” accepted at GECCO 2026. Their framework searches over complete, executable update rules by combining evolutionary computation with large language models as generative variation operators, producing novel RL algorithms that avoid all canonical mechanisms.

Problem Statement and Motivation

The central argument of the paper is that searching over RL update rules is fundamentally incompatible with gradient-based optimization. The search space consists of discrete program structures implementing full training procedures rather than continuous parameters. Even small changes to update logic can induce qualitatively different learning dynamics, and RL algorithms are notoriously sensitive to implementation details [Henderson2018] [Engstrom2020]. Hyperparameters that stabilize one update rule often fail for another [Patterson2024], which prevents effective local refinement. As a result, each candidate update rule must be treated as a distinct algorithm whose quality can only be assessed through complete training runs. Under these conditions, population-based evolutionary methods are a natural fit because they compare candidates using end-to-end performance without requiring gradients or smooth objective landscapes [Jaderberg2017] [CoReyes2021].

The work builds directly on REvolve [Hazra2025], an evolutionary system developed by the same research group at Orebro University that uses large language models as generative variation operators for reward-function discovery. REvolve formulates reward optimization as an island-based evolutionary process, where LLMs generate syntactically valid reward code that is evaluated by training RL agents. Sygkounas et al. [Sygkounas2026] apply the same evolutionary architecture to a fundamentally different and more challenging search space. Instead of evolving reward functions, which are relatively short expressions with graded behavioral effects, they evolve entire learning update rules as executable Python code. Each candidate defines a complete training procedure, including a differentiable loss function $\mathcal{L}_f(\theta, \xi_t; \mathcal{D}_t)$ and all auxiliary state updates, under a fixed policy architecture, optimizer, and training loop. In this formulation, each distinct update rule f defines a distinct RL algorithm.

The Evolutionary Loop

The evolutionary search follows an island-based population model [CantuPaz1998]. For each generation, each island produces 24 new candidate update rules through two variation operators selected stochastically. Macro mutation, applied with probability $p_{\text{macro}} = 0.65$, rewrites exactly one semantically coherent component of an existing update rule in a single step. This choice reflects the finding that micro mutations, which work well for short reward expressions in

REvolve, are insufficient for update rules where deficiencies in early generations are structural rather than local. Diversity-aware crossover, applied with probability $p_{\text{cross}} = 0.35$, combines two parent rules selected via a composite score that balances fitness-proportional selection with structural dissimilarity measured by normalized Levenshtein distance [Levenshtein1966]. The Levenshtein weight $\alpha = 0.5$ prevents crossover from recombining near-duplicate parents, which would collapse population diversity [LehmanStanley2011].

Both operators are implemented through a conditional generative model realized by a large language model, $f' \sim q_{\varphi}(f_1, f_2, \text{op}, \mathcal{R}, \mathcal{E})$, which receives the parent update rules, the selected operation type, summary training metrics, and environment interface specifications. The LLM then outputs a complete executable implementation of a new update rule. Critically, during evolutionary variation, the generation process is constrained to forbid explicit instantiation of canonical RL mechanisms. The search excludes actor-critic decomposition [KondaTsitsiklis1999], bootstrapped value targets, temporal-difference error computation [Sutton1999], and policy gradient methods. These constraints are enforced only during evolution, not during post-evolution analysis, and serve to force the emergence of genuinely nonstandard learning rules.

Each evolutionary run executes for 10 generations. Fitness evaluation is rigorous. Every candidate update rule is trained on a fixed set of five environments (CartPole-v1, MountainCar-v0, Acrobot-v1, LunarLander-v3, and HalfCheetah-v5) using 5 independent random seeds per environment, yielding 25 training runs per candidate. Performance is summarized by the maximum evaluation return achieved during training for each environment, then normalized to the unit interval using environment-specific reference bounds and averaged across environments to produce a single scalar fitness. A candidate is accepted into the island population only if its fitness exceeds the current population mean, and accepted candidates replace the lowest-fitness members.

LLMs as Variation Operators and the Performance Gap

The paper evaluates two large language models as generative operators: GPT-5.2 and Claude 4.5 Opus. For each model, two independent evolutionary runs with different random seeds are conducted. The results reveal a stark performance gap. GPT-5.2 consistently attains higher fitness values throughout evolution, with final-generation fitness in the range $[0.65, 0.69]$ across seeds. Claude 4.5 Opus achieves substantially lower fitness in the range $[0.37, 0.47]$ and does not produce competitive candidates in the final generation [Sygkounas2026]. Both models exhibit monotonic increases in maximum fitness across generations, confirming that the evolutionary loop successfully improves candidate quality over time. However, the magnitude of the gap is notable: update rules evolved with Claude 4.5 Opus are excluded entirely from further analysis because they fail to reach competitive performance levels. This finding underscores that the choice of LLM as a variation operator materially affects the quality of the search, and suggests that code generation capabilities for complex, tightly-coupled training procedures differ substantially across model families.

Post-Evolution Hyperparameter Optimization

The paper introduces a key extension over REvolve: a post-evolution LLM-guided hyperparameter optimization (LLM-HPO) stage. REvolve terminates after evolutionary search without additional tuning, but this is insufficient for update rules because RL algorithms are highly sensitive to internal scalar parameters [Henderson2018] [Engstrom2020]. A fixed parameterization can severely underestimate the quality of an update rule. For each top- K evolved update

rule $\hat{f}$, the LLM is provided with the full implementation and environment specifications and returns bounded numeric intervals $\mathcal{B}_{\hat{f}} = \prod_{j=1}^d [\ell_j, u_j]$ for each internal scalar parameter. A random sweep then uniformly samples parameter vectors from this region, evaluates fitness through full training runs, and retains the best-performing configuration β_i^* for final evaluation.

The Discovered Algorithms

The two highest-performing algorithms, both evolved using GPT-5.2, are Confidence-Guided Forward Policy Distillation (CG-FPD) and Differentiable Forward Confidence-Weighted Planning with Controllability Prior (DF-CWP-CP). Both are structurally novel in that they avoid value functions, policy gradients, and Bellman-style updates entirely, instead relying on planning-derived learning signals.

CG-FPD trains a policy by distilling short-horizon plans from a learned latent dynamics model. A latent encoder maps observations to a compact representation, and a learned dynamics model predicts transitions in latent space. At each update step, the cross-entropy method (CEM) generates candidate action sequences, which are scored by rolling out the dynamics model and accumulating predicted rewards. The best plan identified by CEM serves as a supervised teacher signal: the policy is updated via distillation to match the first action of the plan. No value function is learned, and no temporal-difference targets are computed. The entire learning signal derives from the quality of the planner’s output.

DF-CWP-CP takes a different approach to the same planning-without-values paradigm. It optimizes the policy via differentiable short-horizon world-model rollouts that incorporate confidence-weighted desirability and controllability objectives. A confidence head estimates the reliability of each predicted transition, and a dual-flow regularization mechanism maintains two parallel policy representations that are periodically reconciled. A controllability augmentation term encourages the agent to prefer states where its actions have high influence on future outcomes, biasing exploration toward regions of the state space where learning is most effective.

Empirical Results

The discovered algorithms are evaluated on ten Gymnasium environments, including the five used during evolution and five unseen environments (Walker2d, InvertedPendulum, Reacher, Swimmer, and Pusher), and compared against PPO, A2C, DQN, and SAC baselines using an identical 256×256 MLP policy architecture.

Environment	PPO	A2C	DQN	SAC	CG-FPD	DF-CWP-CP
CartPole	500.0	500.0	500.0	–	500.0	500.0
LunarLander	246.6	246.6	250.1	–	241.2	260.6
MountainCar	–128.1	–134.2	–147.8	–	–105.8	–108.7
Acrobot	–63.5	–213.6	–61.9	–	–90.6	–78.7
HalfCheetah	1579.1	795.9	–	4988.5	2407.9	2103.8
Reacher	–3.15	–6.48	–	–2.05	–2.67	–5.43
Swimmer	95.0	49.1	–	87.8	247.5	219.8
Inv. Pendulum	1000.0	1000.0	–	1000.0	1000.0	1000.0
Walker2d	3163.2	801.5	–	4595.3	1603.8	1297.6
Pusher	–25.5	–32.4	–	–25.5	–27.1	–28.7

Table 1: Maximum evaluation return (mean over 5 seeds) for evolved algorithms vs. baselines. Top five rows are training environments; bottom five are unseen. Bold indicates best performance. Dashes indicate inapplicable baselines (DQN for continuous, SAC for discrete). Adapted from Sygkounas et al. [Sygkounas2026].

The results reveal a nuanced performance profile. On the unseen Swimmer environment, CG-FPD achieves a return of 247.5, dramatically outperforming PPO (95.0), SAC (87.8), and A2C (49.1). DF-CWP-CP also excels on Swimmer at 219.8. On LunarLander, DF-CWP-CP achieves the highest return (260.6) among all methods. Both evolved algorithms match the optimal score on CartPole and InvertedPendulum. On MountainCar, CG-FPD achieves -105.8 , outperforming all baselines including PPO (-128.1) and DQN (-147.8). However, the evolved algorithms fall short on high-dimensional continuous control tasks where SAC dominates. On HalfCheetah, SAC achieves 4988.5 compared to CG-FPD’s 2407.9, and on Walker2d, SAC reaches 4595.3 versus CG-FPD’s 1603.8. This pattern suggests that planning-based update rules without value functions scale less effectively to tasks requiring fine-grained continuous control over many degrees of freedom.

Training Stability and Ablation Studies

A notable limitation of the discovered algorithms is training instability. Evaluation curves show that both CG-FPD and DF-CWP-CP reach high performance within the training budget, but learning is often non-monotonic and performance is not consistently maintained at the end of training [Sygkounas2026]. The reported results rely on best-checkpoint selection rather than final-policy performance, which masks the instability during training.

Two ablation studies provide additional insight. The first examines the Levenshtein regularization weight α in the crossover operator. With $\alpha = 0$ (no structural regularization), maximum fitness reaches approximately 0.50; with $\alpha = 1$ (full regularization), fitness reaches 0.56. The intermediate setting $\alpha = 0.5$ used in the main experiments achieves the highest fitness range of $[0.65, 0.69]$, demonstrating that partial structural regularization yields the best balance between preserving functional code structure and enabling exploratory variation.

The second ablation adds a terminal value bootstrap to CG-FPD via a value head trained with one-step temporal-difference learning, used only as a bounded terminal bonus in the planner score. Across all tested environments, adding this canonical RL mechanism reduces peak performance but consistently lowers variance across seeds. This result is particularly telling: it demonstrates that CG-FPD is not limited by missing long-horizon return estimates,

and that introducing learned value signals actually biases planning and degrades peak performance even when used conservatively. The stability-performance tradeoff suggests that the evolved algorithms occupy a fundamentally different design region than conventional value-based methods.

Computational Cost and Broader Implications

The computational requirements are substantial. All evolutionary experiments were conducted on four NVIDIA A100 GPUs with 40 GB of memory each, and a single evolutionary generation required approximately 30 hours of wall-clock time. The full evolutionary search (10 generations) therefore requires roughly 300 hours of GPU time per run, executed on the LUMI supercomputer. This cost is driven by the requirement that each candidate must be evaluated through 25 complete training runs (5 environments by 5 seeds).

The broader implication of this work is that LLM-guided evolutionary search can discover RL algorithms that are structurally distinct from all existing methods. The discovered algorithms replace the entire value-estimation and policy-gradient machinery of modern RL with planning-based distillation and differentiable world-model rollouts. While the current generation of evolved algorithms does not uniformly surpass established baselines, the framework demonstrates that the space of viable RL update rules extends well beyond what human designers have explored. As noted by the authors, the discovered algorithms arise from novel recombinations of existing RL mechanisms rather than from fundamentally new update forms [Sygkounas2026], reflecting both the structure of the search space and the representational limits of current language models. Future improvements in LLM code generation capabilities and evaluation efficiency may yield algorithms that more consistently outperform hand-designed baselines across diverse environments.

References

- [Sygkounas2026] A. Sygkounas, A. Loutfi, and A. Persson. “Evolutionary Discovery of Reinforcement Learning Algorithms via Large Language Models.” arXiv:2603.28416, 2026. Accepted at GECCO 2026.
- [Hazra2025] R. Hazra, A. Sygkounas, A. Persson, P. Z. dos Santos, A. Loutfi, and S. Dey. “REvolve: Reward Evolution with Large Language Models Using Human Feedback.” ICLR 2025.
- [CoReyes2021] J. D. Co-Reyes, Y. Miao, D. Peng, E. Real, S. Levine, Q. V. Le, H. Lee, and A. Faust. “Evolving Reinforcement Learning Algorithms.” ICLR, 2021.
- [Oh2020] J. Oh, M. Hessel, W. M. Czarnecki, Z. Xu, H. P. van Hasselt, S. Singh, and D. Silver. “Discovering Reinforcement Learning Algorithms.” NeurIPS, 2020.
- [Henderson2018] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger. “Deep Reinforcement Learning that Matters.” AAAI, 2018.
- [Engstrom2020] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry. “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO.” ICLR, 2020.
- [Patterson2024] A. Patterson, S. Neumann, M. White, and A. White. “Empirical Design in Reinforcement Learning.” JMLR, 2024.
- [Jaderberg2017] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, C. Fernando, and K. Kavukcuoglu. “Population Based Training of Neural Networks.” arXiv:1711.09846, 2017.
- [Ma2024] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. “Eureka: Human-Level Reward Design via Coding Large Language Models.” ICLR, 2024.
- [Xu2018] Z. Xu, H. P. van Hasselt, and D. Silver. “Meta-Gradient Reinforcement Learning.” NeurIPS, 2018.
- [ZophLe2017] B. Zoph and Q. V. Le. “Neural Architecture Search with Reinforcement Learning.” ICLR, 2017.

- [KondaTsitsiklis1999] V. R. Konda and J. N. Tsitsiklis. "Actor-Critic Algorithms." NeurIPS, 1999.
- [Sutton1999] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." NeurIPS, 1999.
- [CantuPaz1998] E. Cantu-Paz. "A Survey of Parallel Genetic Algorithms." *Calculateurs Paralleles, Reseaux et Systemes Repartis*, 1998.
- [Levenshtein1966] V. I. Levenshtein. "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals." *Soviet Physics Doklady*, 1966.
- [LehmanStanley2011] J. Lehman and K. O. Stanley. "Evolving a Diversity of Virtual Creatures Through Novelty Search and Local Competition." GECCO, 2011.